SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – FLOWCHART-TO-CODE CONVERSION

Course Code:	CSA0305	**Course Title:**	Data Structures
CO Assessed:	CO5 – Naturalization (BL4, BL6)	**Assessment:**	Assessment Tool 2 – Flowchart-To-Code Conversion
Weightage:	35% of CIA	**Total Marks:**	100
CO5 Statement:	*Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem.*		
Student Name:	Sasikumar Megha	**Reg. No.:**	192471038
Section / Batch:	3rd CSBS	**Date:**	04/09/2026

Code of Conduct

I Sasikumar Megha *(Name / Reg No)*

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S. Megha.

Instructions

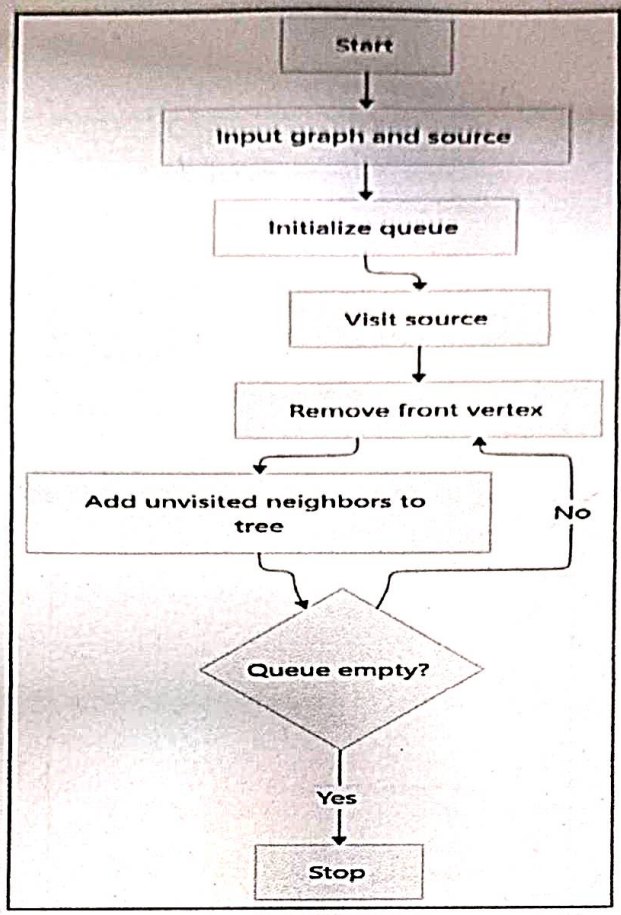
- This test contains 5 Flowchart-to-code conversion questions. Total: 100 Marks.
- When a student answer using Flowchart-to-code conversion should evaluate based on the ability to design clear, logical, and efficient code solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

Section / Topic	Focus	Q Nos	Marks
Graph Traversal	BFS	1	20
Shortest Path Algorithm	Dijkstra's Algorithm	2	20
Shortest Path Algorithm	Unweighted Graph	3	20
Graph Traversal	BFS	4	20
Minimum Spanning Tree	Prim's Algorithm	5	20
	GRAND TOTAL:		**100 Marks**

Annexure A – Pseudocode Writing Task

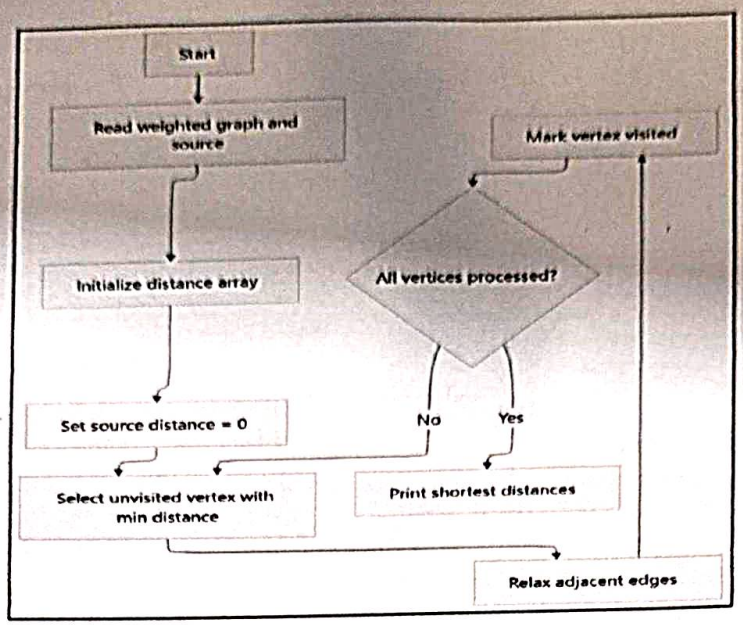
1. Convert the flowchart for generating a BFS tree into code. (20 Marks)

Flowchart Diagram:



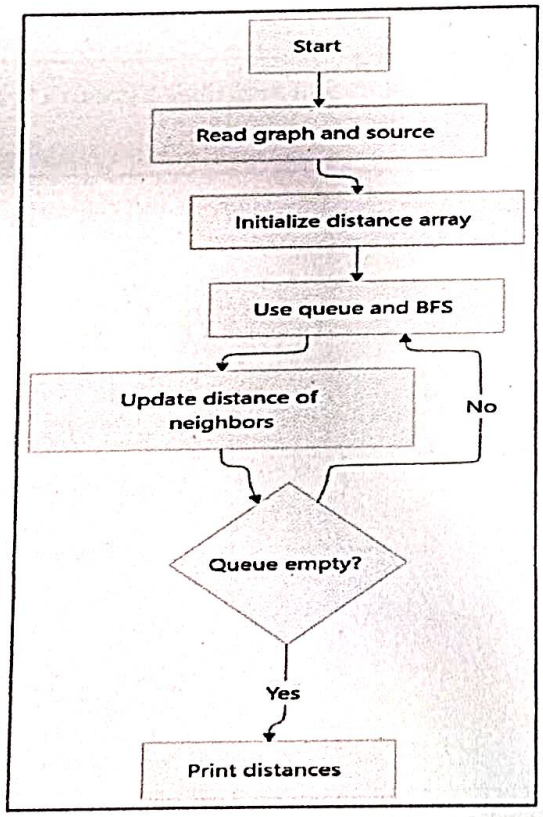
2. Convert the flowchart for Dijkstra's shortest path algorithm into code. (20 Marks)

Flowchart Diagram:



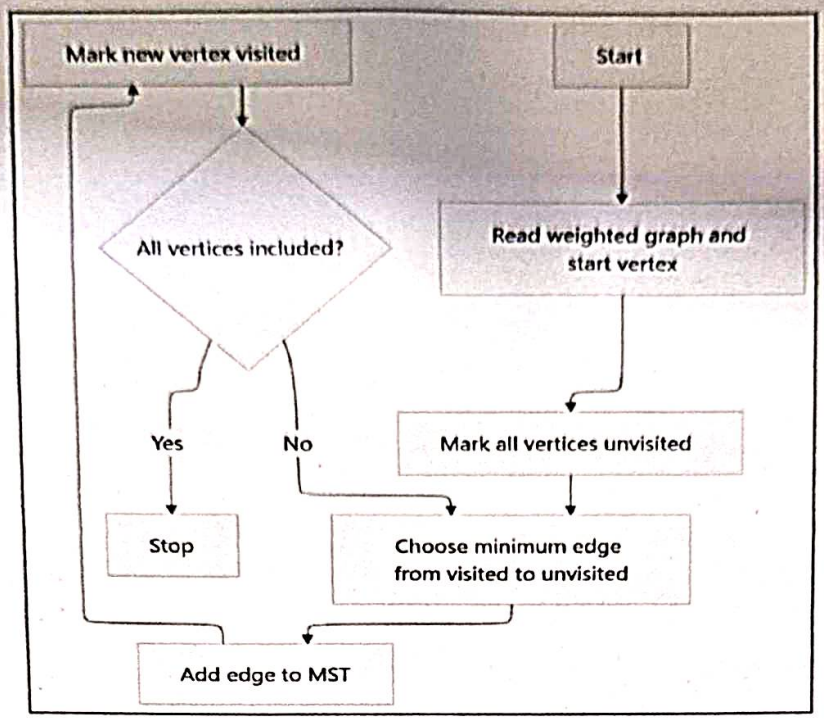
3. Convert the flowchart for shortest path in an unweighted graph into code. (20 Marks)

Flowchart Diagram:



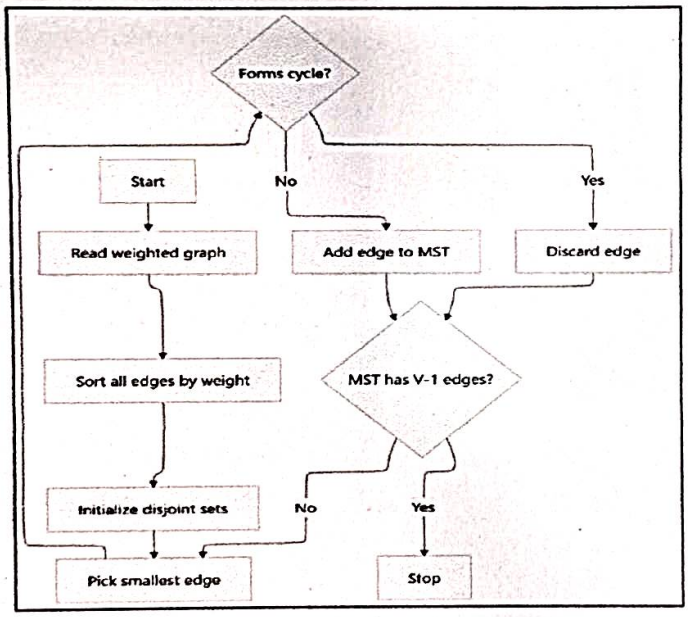
4. **Convert the flowchart for Prim's algorithm into code.** **(20 Marks)**

Flowchart Diagram:



5. Convert the flowchart for Kruskal's algorithm into code. (20 Marks)

Flowchart Diagram:



Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Demonstrates complete and clear understanding; handles edge cases effectively	Good understanding; minor gaps in edge case handling	Partial understanding; misses some requirements	Misinterprets problem; major gaps
Code Correctness	30	Fully correct; passes all test cases	Mostly correct; minor errors	Partially correct; several test cases fail	Incorrect or non-functional code
Code Quality & Readability	20	Clean, modular, well-commented, follows standards	Readable with some structure	Limited readability; poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence

Faculty In-charge	Course Coordinator	Program Director
Signature:	Signature: _____	Signature: _____
Date: 04/9/26	Date: _____	Date: _____

Sasikumar Megha
192471038
CSA 0305

Co_5 - AT 2

1) Breadth first Search (BFS)

```
#include <stdio.h>
#define MAX 20
int main(){
  int graph[MAX][MAX], visited[MAX] = {0};
  int queue[MAX], front = 0, rear = 0;
  int n, source, i, v;
  Printf("Enter number of vertices:");
scanf("%d", &n);
Printf("Enter adjacent matrix:\n");
for(i=0; i<n; i++)
 for(v=0; v<n; v++)
   scanf("%d", &graph[i][v]);
Printf("Enter source vertex:");
scanf("%d", &source);
queue[rear++] = source;
visited[source] = 1;
Printf("BFS Traversal:");
```

```
while (front < rear) {
  int u = queue [front++];
  Printf ("%d", u);
  for (v=0; v<n; v++) {
    if (graph [u][v] == 1 && visited [v] == 0) {
      visited [v] = 1;
      queue [rear ++] = v;
    }
  }
}
return 0;
}.
```

Sample Input

Enter number of vertex : 5

Enter adjacency matrix:

```
0 1 1 0 0
1 0 0 1 0
1 0 0 1 1
0 1 1 0 1
0 0 1 1 0
```

Enter source vertex : 0

Sample output

BFS Traversal : 0 1 2 3 4

2) Dijkstra's shortest Path Algorithm.

```
#include <stdio.h>
#define MAX 20
#define INF 9999
int main(){
  int graph[MAX][MAX], dist[MAX], visited[MAX]={0};
  int n, source, i, j, count, u, min;
Printf("Enter number of vertices: ");
scanf("%d", &n);
Printf("Enter weighted adjacency matrix :\n");
Printf("Enter 0 if there is no edge.\n");
 for(i=0; i<n; i++)
    for(j=0; j<n; j++)
      scanf("%d", &graph[i][j]);
Printf("Enter source vertex: ");
scanf("%d", &source);
 for(i=0; i<n; i++){
  if(graph[source][i] != 0)
    dist[i] = graph[source][i];
  else
   dist[i] = INF;
}
```

```
dist [source] = 0;
for (count = 0; count < n; count++) {
  min = INF;
  u = -1;
for (i=0; i<n ; i++) {
  if (!visited[i] && dist[i] < min) {
    min = dist[i];
     u = i;
  }
}
if (u == -1)
  break;
visited [u] = 1;
for (j=0; j<n ; j++) {
  if (!visited [j] && graph [u][j] != 0 &&
     dist[u] + graph[u][j] < dist [j]) {
    dist [j] = dist [u] + graph [u][j];
  }
}
}
Printf ("\n shortest distance from vertex %d:\n",
        source );
for (i=0 ; i<n; i++) {
  if (dist[i] == INF)
     Printf ("vertex % d : INF \n", i);
else
Printf("vertex %d: %d\n",i,dist[i]);} return 0; }
```

Sample Input

```
enter no. of vertices=5
0  10  3  0  0
10  0  1  2  0
3  1  0  8  2
0  2  8  0  7
0  0  2  7  0
enter source vertex:0
```

output.

```
shortest distance from
  vertex 0:
  vertex 0 : 0
  vertex 1 : 4
  vertex 2 : 3
  vertex 3 : 6
  vertex 4 : 5
```

3) Shortest Path in a Unweighted Graph

```
#include <stdio.h>
#define MAX 20
int main() {
    int graph[MAX][MAX];
    int distance[MAX], visited[MAX] = {0};
    int queue[MAX];
    int front = 0, rear = 0;
    int n, source, i, v, u;
Printf("enter number of vertices: ");
scanf("%d", &n);
Printf("enter adjacency matrix: \n");
for (i=0; i<n; i++)
 for (v=0; v<n; v++)
    scanf("%d", &graph[i][v]);
Printf("Enter source vertex: ");
 scanf("%d", &source);
 for (i=0; i<n; i++)
    distance[i] = -1;
```

```
queue [rear++] = source;
visited [source] = 1;
distance [source] = 0;
while (front < rear) {
  u = queue [front++];
  for (v=0; v<n; v++) {
    if (graph [u][v] == 1 && !visited[v]){
      visited [v] = 1;
      distance [v] = distance [u] + 1;
      queue [rear++] = v;
    }
  }
}
Printf ("\n shortest distance from vertex %d: \n", source);
for (i=0; i<n; i++){
  printf ("vertex %d : %d \n", i, distance [i]);
}
return 0;
}
```

Sample Input:

```
Enter no. of vertices : 5
0 1 1 0 0
1 0 0 1 0
1 0 0 1 1
0 1 1 0 1
0 0 1 1 0
```

Sample Output

```
Shortest distance from vertex 0:
vertex 0 : 0
vertex 1 : 1
vertex 2 : 1
vertex 3 : 2
vertex 4 : 2
```

1) Prim's Minimum Spanning Tree Algorithm

```
#include <studio.h>
#define Max 20
#define INF 9999
int main() {
  int graph[MAX][MAX], visited[MAX] = {0};
  int n, i, j, edges = 0;
  int min, u=0, v=0, total = 0;
Printf("Enter number of vertices :");
Scanf("%d", &n);
Printf("Enter weighted adjacency matrix :\n");
Printf("Enter 0. if there is no edge .\n");
for (i=0 ; i<n ; i++)
    for (j=0 ; j<n ; j++)
        Scanf("%d", &graph[i][j]);
Printf("Enter starting vertex :");
Scanf("%d", &u);
visited[u] = 1;
Printf("\n Edges in minimum spanning Tree :\n");
while (edges <n-1) {
    min = INF;
```

```
for (i=0; i<n; i++){
 if (visited[i]) {
   for (j=0; j<n; j++) {
     if (!visited[j] && graph[i][j] != 0 &&
         graph[i][j] < min) {
       min = graph[i][j];
       u = i;
       v = j; }
   }
 }
 if (min == INF)
   break;
 Printf ("%d - %d : %d \n", u, v, min);
 total += min;
 visited[v] = 1;
 edges ++;
}
Printf ("Total cost = %d \n", total); return 0; }
```

Sample Input

```
Enter no of vertices: 5
0 2 3 0 0
2 0 1 4 0
3 1 0 5 6
0 4 5 0 2
0 0 6 2 0
Enter starting vertex: 0
```

Sample output.

```
Edges in minimum spanning tree
0 - 1 : 2
1 - 2 : 1
1 - 3 : 4
3 - 4 : 2
Total cost = 9
```

Kruskal's Minimum Spanning Tree Algorithm.

```
#include <stdio.h>
#define MAX 100
struct edge {
    int u, v, weight;
};
int parent[MAX];
int find(int x){
  if (parent[x]==x)
    return x;
  return parent[x]=find(parent[x]);
}
void unionset(int a, int b){
    a = find(a);
    b= find(b);
  if (a!= b)
    Parent[b] =a;
}
int main(){
  struct edges edges[MAX], temp;
  int n, e =0;
int i, j, count = 0, total = 0;
Printf("Enter number of vertices:");
scanf("%d", &n);
Printf("Enter number of edges:");
```

```
Scanf("%d", &e);
Printf("enter edges (source destination weight):\n");
for (i=0; i<e; i++){
  for (j=0, j<e-i-1; j++){
   if (edges[j].weight > edges[j+1].weight){
    temp = edges[j];
    edges[j] = edges[j+1];
    edges[j+1] = temp;}
  }}
for (i=0; i<n; i++)
  Parent[i] = i;
Printf("\n Edges in minimum spanning Tree:\n");
for (i=0; i<e && count<n-1; i++){
 int u = edges[i].u;
 int v = edges[i].v;
 if (find(u) != find(v)){
  Printf("%d - %d : %d\n",
          u, v, edges[i].weight);
 total += edges[i].weight;
 union set(u,v);
 count++;
 }}
Printf("Total cost = %d\n", total);
return 0; }
```

Sample Input

```
Enter no. of vertices: 5
Enter no. of edges: 7
enter edges:
0 1 2
0 2 3
1 2 1
1 3 4
2 3 5
2 4 6
3 4 2
```

Sample output

```
Edges in minimum spaning Tree
1 - 2 : 1
0 - 1 : 2
3 - 4 : 2
1 - 3 : 4
Total cost = 9
```